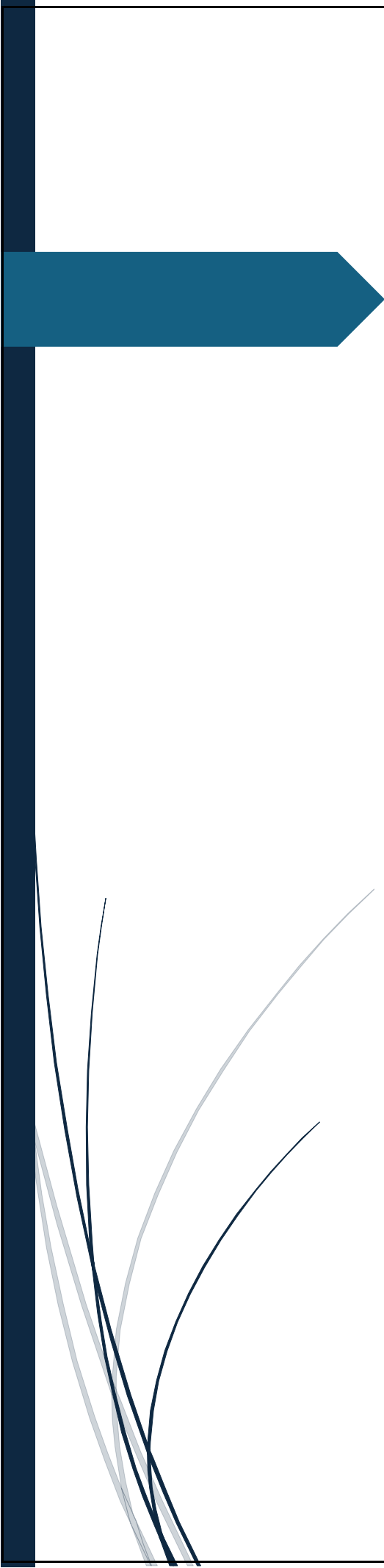




Speech Emotion Recognition

Using RAVDESS, CREMA-D, TESS
AND SAVEE datasets

Syeda Ayesha Wajahat

Speech Emotion Recognition using	4
Deep Learning	4
Abstract	4
Introduction.....	4
1. Background and Motivation	4
2. Objective	4
3. Key Contributions.....	4
Libraries	5
Dataset Description	6
1. RAVDESS	6
2. CREMA-D	7
3. TESS	10
4. SAVEE.....	12
5. Dataset Combination and Balance.....	14
6. Class Distribution.....	14
Data Preprocessing and Augmentation	14
1. Audio Loading and Trimming	15
Audio Visualization Using Waveplot and Spectrogram	15
Purpose of Short-Term Fourier Transform (STFT):	17
2. Data Augmentation Techniques	18
noise(data, random=False, rate=0.035, threshold=0.075).....	18
stretch(data, rate=0.8)	19
shift(data, rate=1000).....	19
pitch(data, sampling_rate, pitch_factor=0.7, random=False).....	19
Audio Visualization after Augmentation:.....	20
3. Feature Extraction	21
Audio Features for Signal Processing	22
4. Feature Matrix Construction	23
Reasons for Selecting ZCR, RMS, and MFCC	24
Feature Scaling and Dataset Splitting	25
1. Label Encoding.....	25
2. Dataset Splitting Strategy.....	26
3. Feature Normalization (Standard Scaling)	27
4. Input Reshaping for 1D CNN	27

Model Architecture and Training	27
1. Choice of Model: 1D Convolutional Neural Network (1D CNN)	28
2. Architectural Overview	28
3. Compilation Details	29
4. Regularization Techniques	29
5. Training Details	30
Training Objective	31
Evaluation and Results	32
1. Test Set Evaluation	32
2. Confusion Matrix	33
3. Classification Report	34
4. Performance Insights	35
Conclusion	36
• Multi-Dataset Integration:	36
• Minimal but Powerful Features:	36
• Effective Augmentation:	36
• CNN Performance:	36
• Evaluation Metrics:	36
Limitations	36
• Speaker Dependence:	36
• Emotion Ambiguity:	36
• No Contextual Understanding:	37
Future Work	37
1. Multimodal Fusion:	37
2. Temporal Models:	37
3. End-to-End Audio Models:	37
4. Real-Time Emotion Detection:	37
5. Transfer Learning & Fine-Tuning:	37
Final Conclusion:	37

Speech Emotion Recognition using Deep Learning

Abstract

This project explores the application of deep learning techniques to **Speech Emotion Recognition (SER)** using audio signals from multiple publicly available datasets. A robust pipeline is developed that includes data preprocessing, augmentation, feature extraction, and classification using a **1D Convolutional Neural Network (CNN)**. Audio data is transformed into numerical features such as **Zero Crossing Rate (ZCR)**, **Root Mean Square Energy (RMS)**, and **Mel-Frequency Cepstral Coefficients (MFCCs)**. These features are fed into a CNN designed for sequential audio analysis. Experimental results show high accuracy and generalization capability across a diverse set of emotions. This demonstrates the effectiveness of combining audio feature engineering with deep learning models for emotion classification tasks.

Introduction

1. Background and Motivation

Human speech conveys more than just words—it reflects a wide spectrum of emotions through tone, pitch, loudness, and rhythm. The ability for machines to recognize such emotions is critical for building emotionally intelligent systems in domains such as virtual assistants, healthcare, customer service, and human-computer interaction. Speech Emotion Recognition (SER) aims to automatically identify the emotional state of a speaker from audio signals.

2. Objective

The primary goal of this project is to design and implement a deep learning pipeline that can accurately classify speech emotions from audio samples using signal processing techniques and a 1D Convolutional Neural Network.

3. Key Contributions

- Combined and preprocessed audio data from four widely-used emotion datasets: **RAVDESS**, **CREMA-D**, **TESS**, and **SAVEE**.
- Applied **audio augmentation techniques** including noise injection and pitch shifting to enhance generalization.
- Extracted compact and discriminative features like **ZCR**, **RMS**, and **MFCCs** from audio clips.
- Trained a deep **1D CNN model** to learn patterns in the extracted features.

- Evaluated model performance using **confusion matrix**, **classification report**, and **accuracy metrics** on unseen test data.

Libraries

This cell loads the Python libraries needed for audio processing, data handling, and deep learning model development. Key imports include:

- **librosa** and **librosa.display**:
Used for audio analysis and visualization, such as plotting waveforms and spectrograms.
- **matplotlib.pyplot** and **seaborn**:
For creating various plots and graphs, such as the distribution of emotion classes.
- **numpy** and **pandas**:
For numerical computations and working with data in tabular (DataFrame) format.
- **IPython.display.Audio**:
Enables playback of audio clips directly within the notebook.
- **keras** (from TensorFlow):
Used for building and training deep learning models, including layers and model structures.
- **keras.utils.np_utils**:
Provides utility functions such as one-hot encoding of labels.
- **sklearn.model_selection.train_test_split**:
Splits the dataset into training, validation, and test sets.
- **sklearn.preprocessing.LabelEncoder**:
Converts categorical emotion labels into numerical form.
- **sklearn.preprocessing.StandardScaler**:
Normalizes feature data for improved model performance.
- **EarlyStopping** and **ReduceLROnPlateau** (Keras callbacks):
Help control training by stopping early if validation loss stops improving and reducing the learning rate when the model plateaus.

Dataset Description

This project utilizes four benchmark datasets for emotion classification from speech audio. Each dataset brings diversity in speakers, expressions, and recording conditions, enabling a more generalizable model.

1. RAVDESS

(Ryerson Audio-Visual Database of Emotional Speech and Song)

- **Actors:** 24 (12 male, 12 female)
- **Emotions Covered:** Neutral, Calm, Happy, Sad, Angry, Fearful, Disgust, Surprised
- **Filename Encoding:** Encodes modality, emotion, intensity, statement, repetition, actor
- **Label Extraction:** Emotion codes parsed from the file name (e.g., 02-01-06-01-02-01-12.wav)

	Emotion	Path
0	surprise	../input/Ravdess/audio_speech_actors_01-24/Act...
1	neutral	../input/Ravdess/audio_speech_actors_01-24/Act...
2	disgust	../input/Ravdess/audio_speech_actors_01-24/Act...
3	disgust	../input/Ravdess/audio_speech_actors_01-24/Act...
4	neutral	../input/Ravdess/audio_speech_actors_01-24/Act...

Filenames

Here is the filename identifiers as per the official RAVDESS website:

- Modality (01 = full-AV, 02 = video-only, 03 = audio-only).
- Vocal channel (01 = speech, 02 = song).
- Emotion (01 = neutral, 02 = calm, 03 = happy, 04 = sad, 05 = angry, 06 = fearful, 07 = disgust, 08 = surprised).
- Emotional intensity (01 = normal, 02 = strong). NOTE: There is no strong intensity for the 'neutral' emotion.

- Statement (01 = "Kids are talking by the door", 02 = "Dogs are sitting by the door").
- Repetition (01 = 1st repetition, 02 = 2nd repetition).
- Actor (01 to 24. Odd numbered actors are male, even numbered actors are female).

So, here's an example of an audio filename. 02-01-06-01-02-01-12.mp4 This means the meta data for the audio file is:

- Video-only (02)
- Speech (01)
- Fearful (06)
- Normal intensity (01)
- Statement "dogs" (02)
- 1st Repetition (01)
- 12th Actor (12) - Female (as the actor ID number is even)

Filename Breakdown: 02-01-06-01-02-01-12.mp4

Segment	Meaning	Value	Description
02	Modality	02	Video-only (not audio-only or full AV)
01	Vocal Channel	01	Speech (not song)
06	Emotion	06	Fearful emotion
01	Emotional Intensity	01	Normal intensity (not strong)
02	Statement	02	"Dogs are sitting by the door"
01	Repetition	01	First repetition of the statement
12	Actor	12	Actor #12 (even number → Female voice)

2. CREMA-D

(Crowd-Sourced Emotional Multimodal Actors Dataset)

- **Actors:** 91 (diverse in gender, ethnicity, age)

- **Emotions Covered:** Neutral, Happy, Angry, Sad, Fearful, Disgust
- **Label Extraction:** Based on 3-letter emotion tags in file names (e.g., 1001_DFA_DIS_XX.wav)

	Emotion	Path
0	disgust	../input/Crema/1028_TSI_DIS_XX.wav
1	happy	../input/Crema/1075_IEO_HAP_LO.wav
2	happy	../input/Crema/1084_ITS_HAP_XX.wav
3	disgust	../input/Crema/1067_IWW_DIS_XX.wav
4	disgust	../input/Crema/1066_TIE_DIS_XX.wav

Filenames

For CREMA, filenames have an underscore-delimited format where the emotion appears as a 3-letter code (SAD, ANG, HAP, DIS, FEA, NEU, or xxx for uninterpreted). We:

List all WAV files in the CREMA directory.

Split each filename by _ to get components, the third part (info[2]) is the emotion code.

Use a series of if statements to map these codes to emotion labels ("sad", "angry", etc.).

If an audio file does not match known codes, label it "unknown" (though CREMA files should match one of the known codes).

Filename: 1001_IE0_DIS_HI.wav

Segment	Meaning
1001	Actor ID
IE0	Utterance type (Improvised Emotional Overact)
DIS	Emotion (Disgust)
HI	Emotion intensity = High

Emotion Codes

Code	Emotion
ANG	Anger
DIS	Disgust
FEA	Fear
HAP	Happy
NEU	Neutral
SAD	Sadness

Intensity Levels

Code	Intensity
LO	Low
MD	Medium
HI	High
XX	Not specified

Detailed Description of Utterance Types in CREMA-D

Code	Full Form / Type	Detailed Description	
DFA	Directed Facial Action	Scripted sentences with actors instructed to show strong and clear emotional expressions. Used to create highly expressive, controlled data.	
IEO	Improvised Emotional Overact	Actors improvised their speech with exaggerated emotional expression. Used to mimic dramatic emotional speech.	
IOM	Improvised Emotional Moderate	Improvised speech with moderate emotional intensity. More natural and less dramatic than IEO.	
ITH	Improvised Thoughtful	The actor speaks slowly and thoughtfully, expressing emotion in a pensive, reflective tone.	
ITS	Improvised Thoughtful Subtle	A more subtle and quiet form of ITH, used to capture low-intensity emotional cues.	
IWL	Improvised With Laugh	Actor adds a laugh to their speech, either naturally or to show an emotion like nervousness or joy.	
IWW	Improvised While Whispering	The actor whispers their speech, used to capture low-volume, intimate, or fearful emotions.	
MTI	Monotone Intentional	The actor intentionally speaks in a flat, monotone voice. Used as a baseline or control condition.	
TAI	Textual Acting Intentional	Actor performs scripted lines with a clear intentional emotional expression, typically stronger than TSI.	
TIE	Textual Improvised Emotion	A mix of text-based content with improvised emotional delivery (not strictly scripted).	
TSI	Textual Scripted Implicit	Scripted lines where emotional expression is more implicit, not exaggerated. Used for subtle emotions.	
WSI	Whispered Scripted Implicit	Scripted lines delivered in a whisper, with implicit emotional cues. Very useful for low-energy emotion modeling.	

3. TESS

(Toronto Emotional Speech Set)

- **Actors:** 2 female speakers
- **Emotions Covered:** Anger, Disgust, Fear, Happiness, Neutral, Surprise, Sadness
- **Structure:** Folders categorized by emotion label; filenames include emotion tags (e.g., OAF_happy_ps.wav)

	Emotion	Path
0	fear	../input/Tess/YAF_fear/YAF_home_fear.wav
1	fear	../input/Tess/YAF_fear/YAF_youth_fear.wav
2	fear	../input/Tess/YAF_fear/YAF_near_fear.wav
3	fear	../input/Tess/YAF_fear/YAF_search_fear.wav
4	fear	../input/Tess/YAF_fear/YAF_pick_fear.wav

Filename:

The TESS dataset folders contain audio files named like OAF_happy_ps.wav where ps stands for “puzzled” (often interpreted as surprise) and other two-letter emotion codes (an angry, sd sadness, etc.). This code:

Lists subdirectories in the TESS folder (e.g., male/female speaker codes).

Iterates through files in each directory.

Splits filenames by _ and takes the third element info[2] as the two-letter emotion code.

If the code is "ps", we map it to "surprise", otherwise use the code as the label.

Speaker Prefixes

Prefix	Speaker
OAF	Older Adult Female
YAF	Younger Adult Female

Emotion Folders

The subfolders under each speaker represent the **emotion** being expressed. These include:

Folder Name	Emotion
angry	Angry
disgust	Disgust
fear	Fear
happy	Happy
neutral	Neutral
sad	Sad
pleasant_surprise	Pleasant Surprise

4. SAVEE

(Surrey Audio-Visual Expressed Emotion)

- **Actors:** 4 male speakers
- **Emotions Covered:** Anger, Disgust, Fear, Happiness, Neutral, Surprise, Sadness
- **Label Extraction:** Based on prefix letters in filenames (e.g., DC_a11.wav)

	Emotion	Path
0	happy	../input/Savee/JE_h09.wav
1	fear	../input/Savee/KL_f12.wav
2	happy	../input/Savee/DC_h03.wav
3	disgust	../input/Savee/DC_d04.wav
4	angry	../input/Savee/KL_a14.wav

Filenames:

The audio files in this dataset are named in such a way that the prefix letters describes the emotion classes as follows:

- 'a' = 'anger'
- 'd' = 'disgust'
- 'f' = 'fear'
- 'h' = 'happiness'
- 'n' = 'neutral'
- 'sa' = 'sadness'
- 'su' = 'surprise'

In SAVEE, filenames start with one or two letters indicating emotion ('a'=anger, 'd'=disgust, 'f'=fear, 'h'=happy, 'n'=neutral, 'sa'=sadness, 'su'=surprise).

For each filename, split on "" and take the second part (split("")[1]) which begins with the emotion letter(s).

Use a regular expression `re.split(r"[0-9]", info)` to strip off any trailing digits, leaving just the letters.

Map the letter(s) to full emotion names via if statements.

Append (emotion, path) to `emotion_df`.

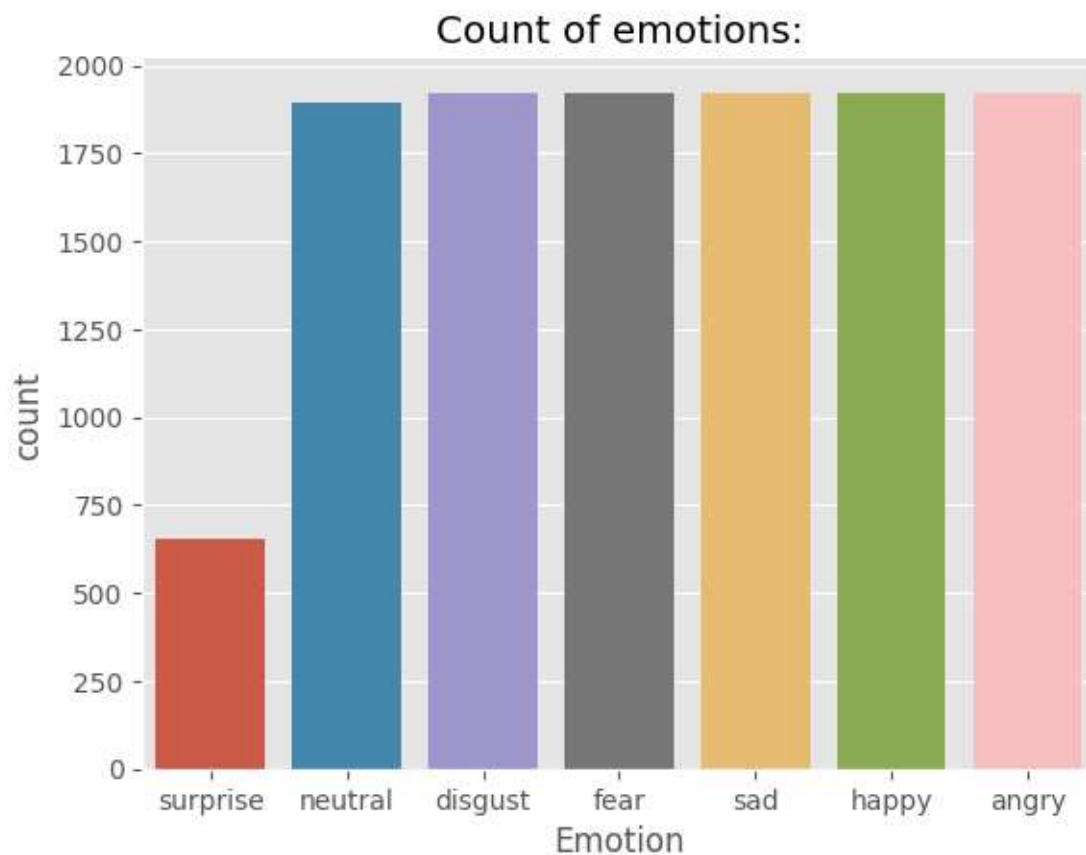
Component	Meaning
DC	Speaker ID (one of the 4 males)
_h13	Emotion + utterance number
h	Emotion code = happiness
13	Sentence/utterance number

5. Dataset Combination and Balance

All four datasets were loaded, parsed for emotion labels, and combined into a single DataFrame. Basic analysis showed some imbalance in emotion class distribution. To mitigate this, **data augmentation** was applied (explained later).

6. Class Distribution

A count plot was generated to visualize the distribution of each emotion category across the combined dataset:



Data Preprocessing and Augmentation

Emotion recognition from audio requires careful preparation of raw signals to ensure consistency, robustness, and generalization. This section outlines the key preprocessing steps and augmentation techniques applied to the dataset.

1. Audio Loading and Trimming

- Each audio file is loaded using **Librosa**, standardized to a **2.5-second duration** with a **0.6-second offset**.
- The offset helps skip initial silence and non-informative speech.

Audio Visualization Using Waveplot and Spectrogram

To get a sense of what the raw audio looks and sounds like, we define helper functions and visualize example samples.

Waveplot and Spectrogram functions

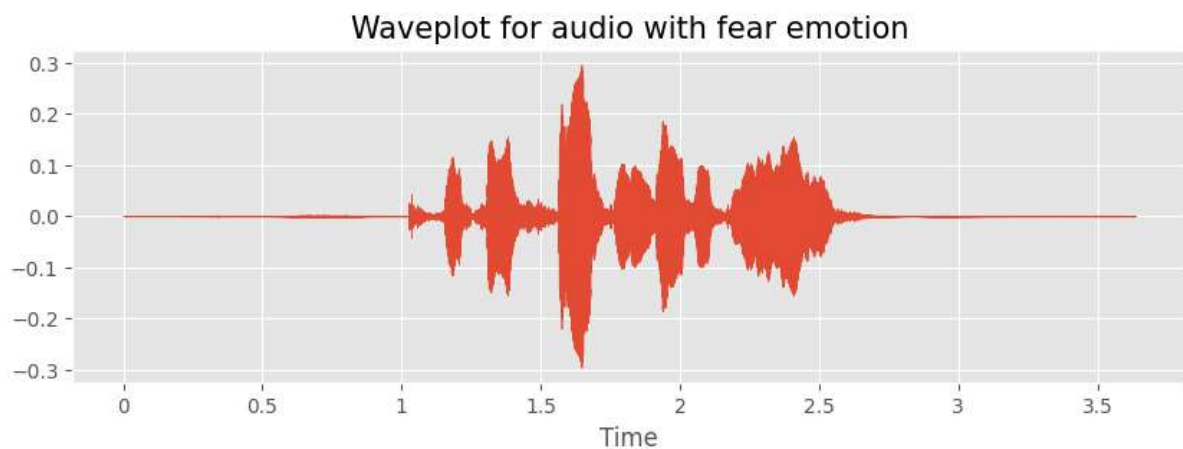
```
create_waveplot(data, sr, e)
```

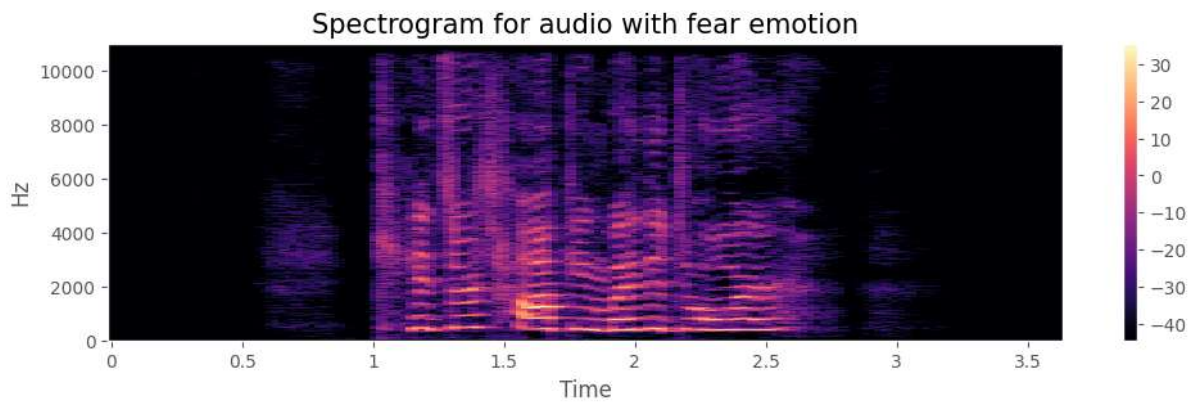
This function displays the waveform plot of the input audio signal with a title indicating the emotion label.

```
create_spectrogram(data, sr, e)
```

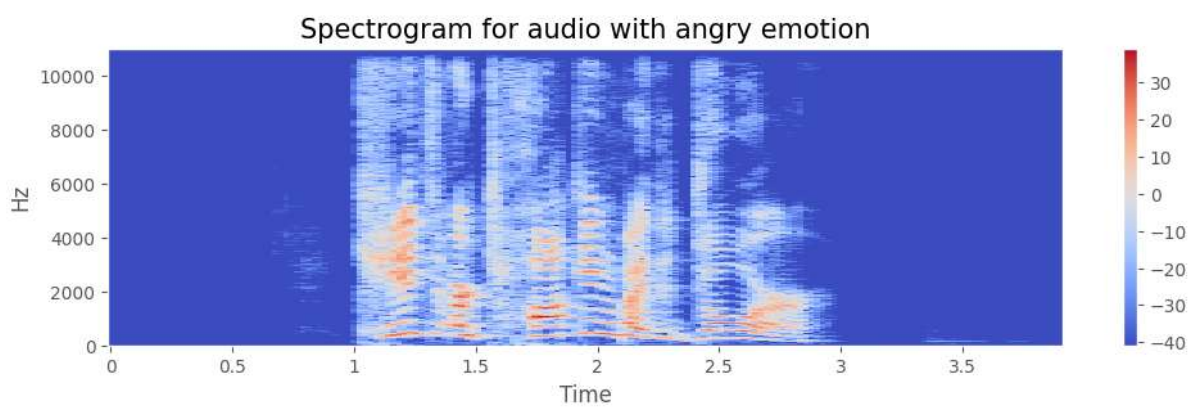
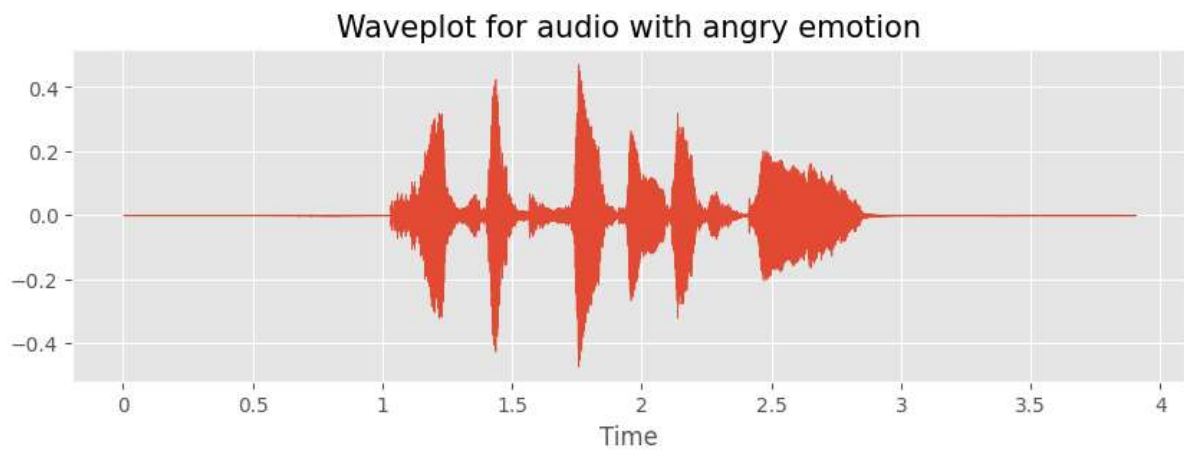
This function computes the STFT of the audio signal, converts the amplitude to decibels, and displays a spectrogram with time on the x-axis and frequency on the y-axis.

Fear Audio Visualization:

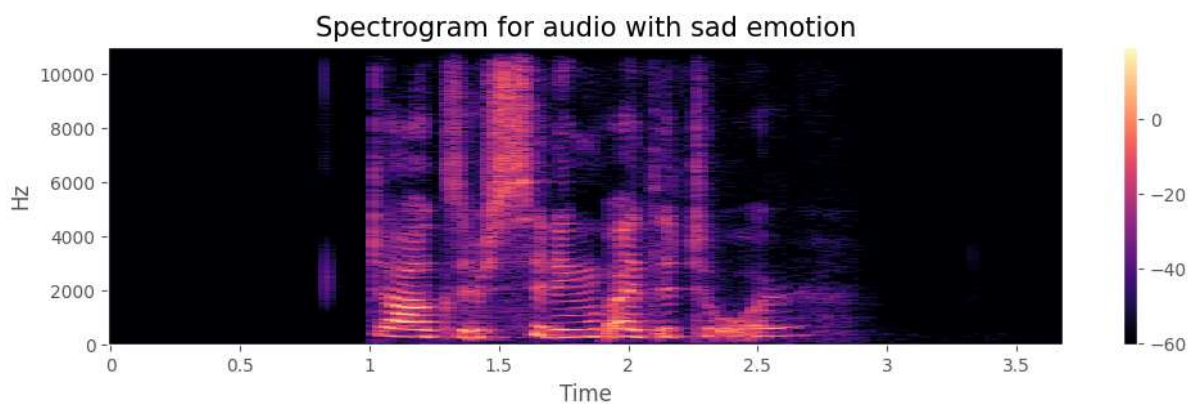
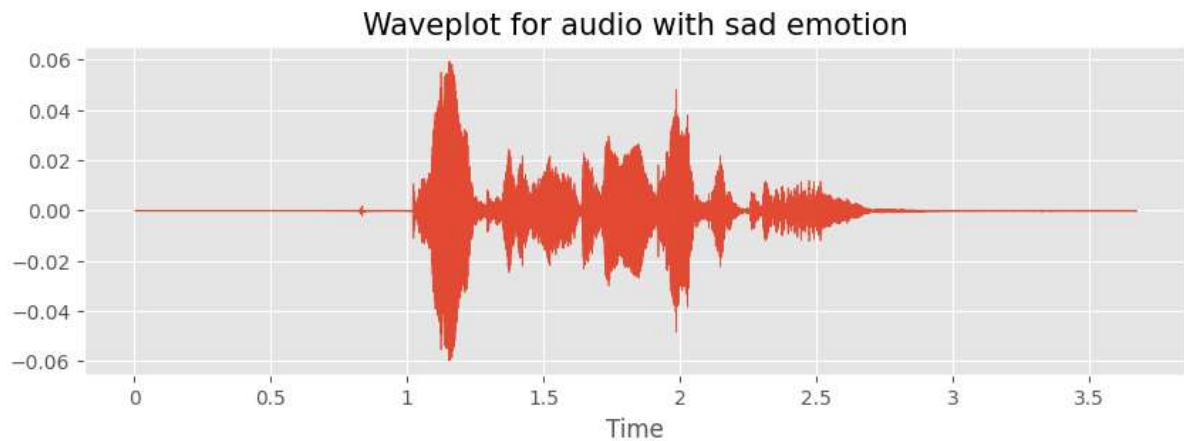




Angry Audio Visualization:



Sad Audio Visualization:



Purpose of Short-Term Fourier Transform (STFT):

The **Short-Term Fourier Transform (STFT)** is used to analyze how the frequency content of a signal changes over time.

- **Why is this needed?**

A regular Fourier Transform (FT) gives you the frequency content of the *entire* signal but loses all time information. For many real-world signals like speech, music, or audio emotions, frequency components vary over time — you want to see *when* certain frequencies appear or disappear.

- **How does STFT work?**

STFT breaks the signal into small overlapping time windows (short segments), and then applies the Fourier Transform to each window separately. This gives a sequence of frequency spectra corresponding to different time intervals.

- **Result:**

You get a **time-frequency representation** — a spectrogram — that shows how frequencies evolve over time.

2. Data Augmentation Techniques

To artificially expand the dataset and introduce variability, four audio augmentations were performed for each file:

We have some ways for data augmentation in sound data:

1. Noise injection
2. Stretching
3. Shifting
4. Pitching

Augmentation	Description
Original	Raw audio without modification.
Noise Injection	Gaussian noise added to simulate recording imperfections.
Pitch Shifting	Shifts the audio pitch slightly up or down.
Combined	Both noise and pitch applied together.

These augmentations help the model learn invariant features and improve generalization.

noise(data, random=False, rate=0.035, threshold=0.075)

Purpose:

Adds artificial noise to the audio data, typically for data augmentation to improve model robustness.

Explanation:

- Adds Gaussian noise to the audio signal.

Parameters:

- random: If True, a random noise level is chosen up to the given threshold.
- rate: If random=False, a fixed noise level is applied.
- threshold: Maximum noise level when random=True.

How it works:

- Calculates noise_amp as a fraction of the audio's maximum amplitude.

- Adds random Gaussian noise scaled by noise_amp to the original audio signal.

stretch(data, rate=0.8)

Purpose:

Stretches or compresses the audio in time without changing its pitch.

Explanation:

- Uses librosa.effects.time_stretch() to modify the duration of the audio.

Parameters:

- rate: Stretch factor.
 - A value less than 1 slows down the audio.
 - A value greater than 1 speeds up the audio.

shift(data, rate=1000)

Purpose:

Shifts the audio waveform forward or backward in time, used to create variation in timing.

Explanation:

- Shifts the waveform by rolling it left or right using np.roll().

Parameters:

- rate: A scaling factor to determine the amount of shift.

How it works:

- A random shift value is selected in the range $[-5 * \text{rate}, 5 * \text{rate}]$.
- The audio data is rolled (circular shift) by this amount.

pitch(data, sampling_rate, pitch_factor=0.7, random=False)

Purpose:

Changes the pitch of the audio without affecting its duration, useful for pitch-based augmentation.

Explanation:

- Uses librosa.effects.pitch_shift() to alter the pitch.

Parameters:

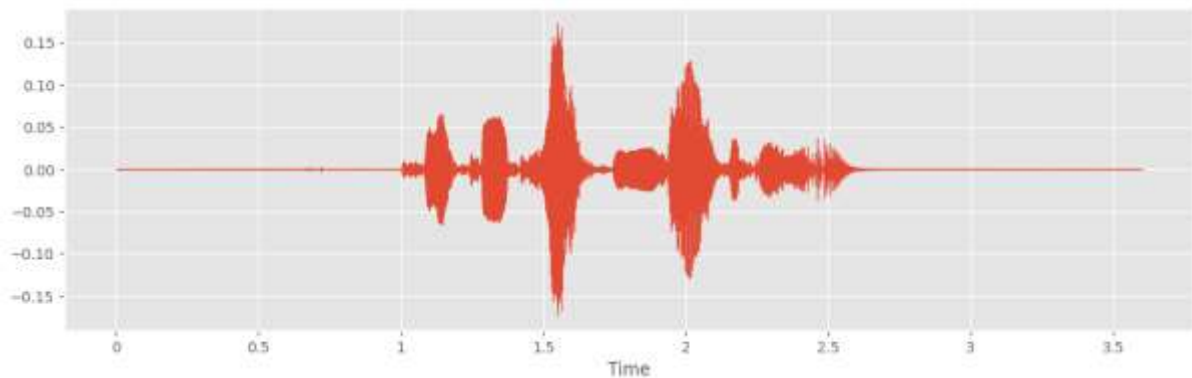
- `sampling_rate`: Sampling rate of the audio.
- `pitch_factor`: Number of semitones to shift the pitch.
- `random`: If True, the pitch shift is chosen randomly up to `pitch_factor`.

How it works:

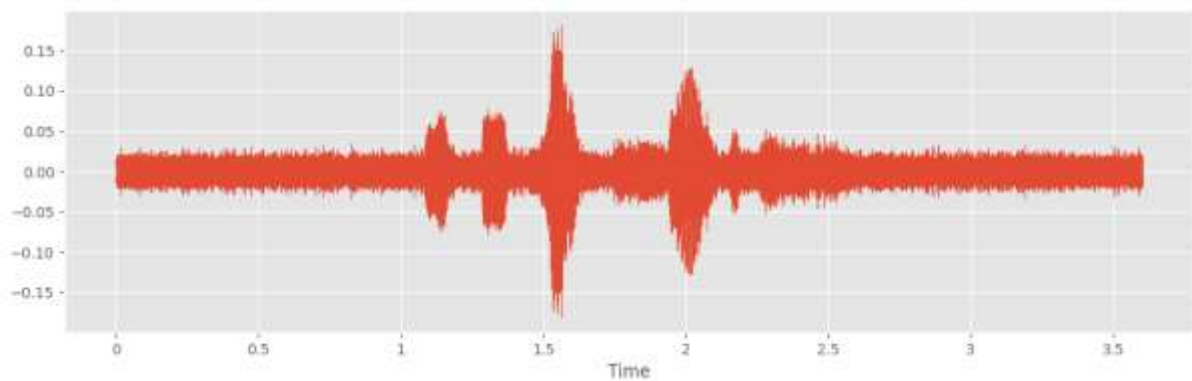
- If `random=True`, a random pitch shift is computed.
- The pitch of the audio is then shifted by the specified number of semitones.

Audio Visualization after Augmentation:

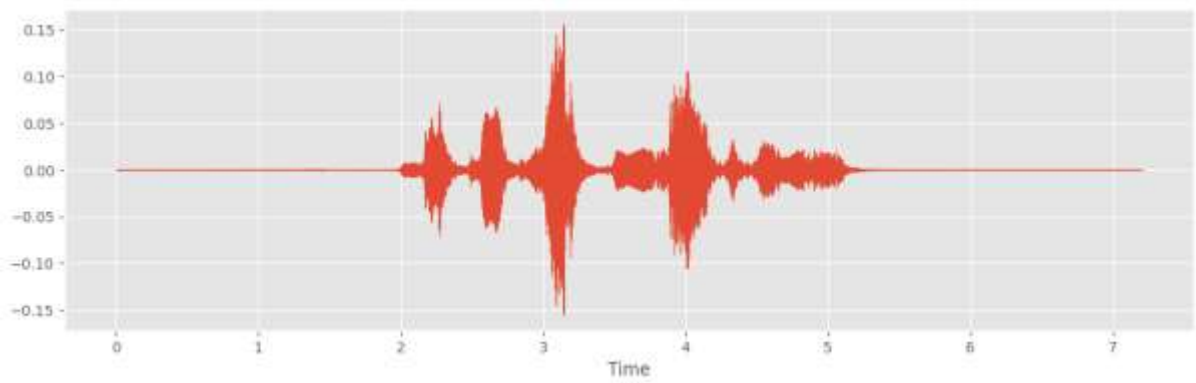
Simple Audio:



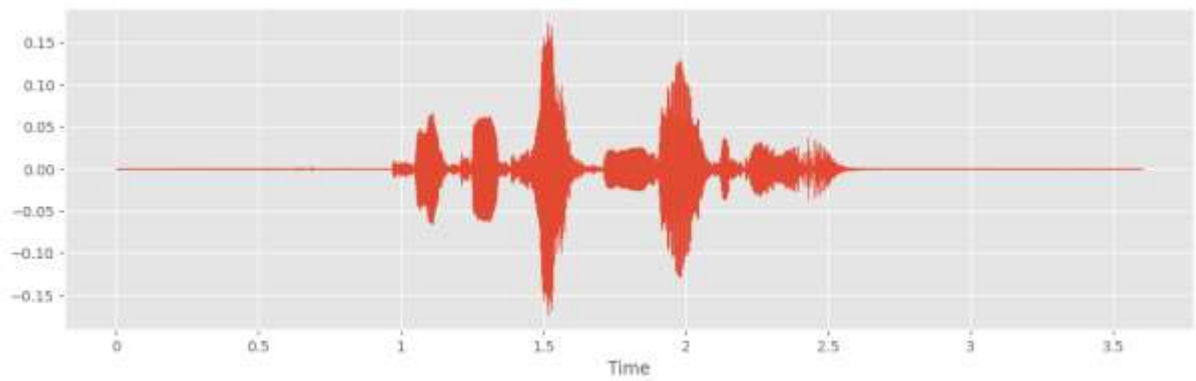
Noised Audio:



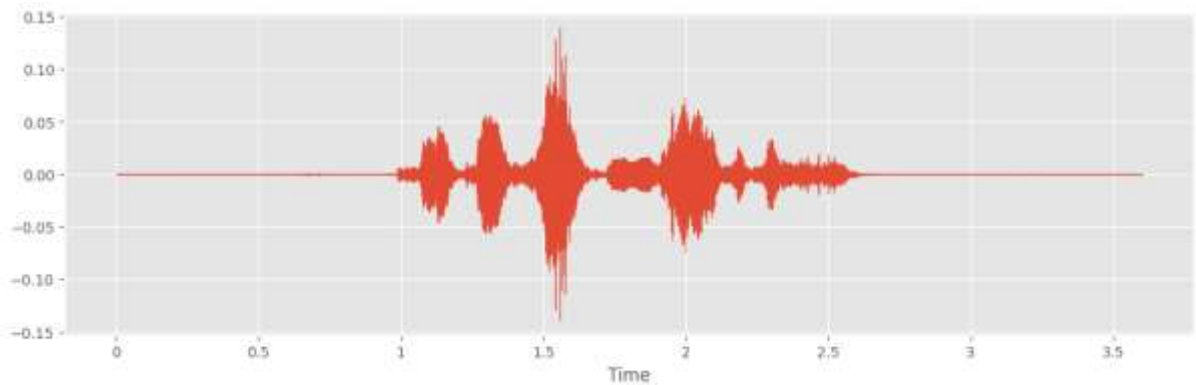
Stretched Audio:



Shifted Audio:



Pitching



3. Feature Extraction

Instead of feeding raw audio, we extract **compact and expressive features**:

Feature	Description
ZCR (Zero Crossing Rate)	Measures signal sign changes, useful for detecting voiced/unvoiced regions.
RMS (Root Mean Square Energy)	Indicates signal loudness/intensity.
MFCC (Mel-Frequency Cepstral Coefficients)	Captures timbral and phonetic characteristics, crucial for speech analysis.

- Total feature vector dimension: **ZCR + RMS + MFCCs = 2376** features per sample.
- MFCCs were selected over other spectral features based on empirical performance and interpretability.

Audio Features for Signal Processing

1. **Zero Crossing Rate**

The rate at which the signal changes sign (i.e., crosses the zero axis) within a given frame. It is often used to characterize noisiness or percussive elements in audio.

2. **Energy**

The total signal energy, calculated as the sum of squared signal values, normalized by the frame length. This feature indicates signal intensity or loudness.

3. **Entropy of Energy**

A measure of the unpredictability or abruptness in energy across sub-frames. High entropy typically indicates more transient or irregular sound patterns.

4. **Spectral Centroid**

Represents the "center of mass" of the spectrum. It is a perceptual measure that indicates where the bulk of the spectral energy is located (e.g., low vs. high frequencies).

5. **Spectral Spread**

The second central moment of the spectrum. It describes the dispersion or variance around the spectral centroid, helping distinguish between narrow-band and broad-band signals.

6. **Spectral Entropy**

The entropy (or complexity) of the normalized spectral energy distribution across sub-frames. It quantifies how flat or peaky the spectrum is.

7. **Spectral Flux**

Measures the rate of change in the power spectrum between consecutive frames. It captures how quickly the frequency content evolves over time.

8. **Spectral Rolloff**

The frequency below which a certain percentage (commonly 85% or 90%) of the total spectral energy is contained. It is used to distinguish between harmonic and noisy signals.

9. **MFCCs (Mel-Frequency Cepstral Coefficients)**

A set of coefficients that represent the short-term power spectrum of a sound. MFCCs are computed using a Mel-scale, which mimics the human ear's perception of sound, making them highly useful in speech and emotion recognition tasks.

Frame Length:

Determines the number of audio samples used in each analysis window (i.e., how much of the signal is processed at once).

Hop Length:

Specifies the number of samples to move forward for the next frame, controlling the overlap between consecutive frames.

4. Feature Matrix Construction

For each audio file, 4 sets of features (original + 3 augmentations) were extracted:

- Features saved into a features.csv file
- Labels encoded as string class names

Resulting shape:

- Features matrix: $X.shape = (\text{num_samples}, 2376)$
- Labels vector: $Y.shape = (\text{num_samples},)$

Feature Name	Shape	Meaning
ZCR	(108,)	1 value per frame → 108 frames
Energy	(108,)	1 value per frame
Entropy of Energy	(108,)	1 value per frame (entropy per sub-frame)
RMS	(108,)	1 value per frame
Spectral Centroid	(108,)	1 value per frame (frequency center of mass)
Spectral Flux	()	Empty → something's wrong with the data passed (likely shape mismatch)
Spectral Rolloff	(108,)	1 value per frame
Chroma STFT	(1296,)	12 chroma bins × 108 frames → flattened to 12×108 = 1296
MelSpectrogram	(13824,)	128 mel bands × 108 frames → flattened → 13824
MFCC	(2160,)	20 MFCCs × 108 frames → flattened → 2160

Reasons for Selecting ZCR, RMS, and MFCC

Complementary Information:

- ZCR captures temporal structure (voicing).
- RMS captures energy or loudness.
- MFCC captures spectral and phonetic content.

Redundancy Avoidance:

- Other features like Energy, Spectral Centroid, etc., provide similar information already covered by RMS or MFCC.

Computational Efficiency:

- Fewer features mean faster feature extraction and model training.

Better Generalization:

- Simplifying the feature set helps avoid overfitting, especially with limited data.

Empirical Validation:

- Experimental results showed that adding more features did not significantly improve performance.

Interpretability:

- These features are well-understood and widely used in speech processing, making them easier to analyze and debug.

In experimental way was decided to use just 3 main features for this task: *ZCR*, *RMS* and *MFCC*.

Also in experimental way was decided to use just 2.5s duration with 0.6 offset - in the dataset first 0.6s contains no information about emotion, and most of them are less then 3s.

Feature Scaling and Dataset Splitting

To ensure optimal performance of the deep learning model and prevent data leakage, a rigorous pipeline of feature normalization, label encoding, and dataset partitioning was followed.

1. Label Encoding

Machine learning models require numerical labels rather than strings. Therefore, the emotion labels (e.g., “happy”, “sad”) were encoded in two stages:

- **Step 1:**
Applied LabelEncoder to transform emotion strings into integer classes.
- **Step 2:**
Used `to_categorical` to convert the integer labels into one-hot encoded vectors for multi-class classification with categorical cross-entropy loss.

Before training a model, we need numeric labels. We use LabelEncoder to map each emotion string to an integer, then convert to a one-hot vector for multi-class classification:

Create a LabelEncoder instance `lb`.

Call `lb.fit_transform(Y)` to convert string labels to integers (0 through 6 for 7 emotions).

Use `np_utils.to_categorical(...)` to convert integer labels into one-hot encoded vectors `Y`.

Print `lb.classes_` to see the order of classes (this shows ['angry', 'disgust', 'fear', 'happy', 'neutral', 'sad', 'surprise']).

Y becomes a 2D array of shape (num_samples, num_classes).

Classes Identified:

Each output label became a 7-dimensional vector, where the index corresponding to the true emotion was marked as 1, and others as 0.

2. Dataset Splitting Strategy

To evaluate generalization performance and prevent overfitting, the feature dataset was split into three subsets:

Subset	Percentage	Purpose
Training	72%	Model learning
Validation	8%	Hyperparameter tuning, early stopping
Test	20%	Final performance evaluation

Splitting was done in two phases:

- **Phase 1:**
Split into train (80%) and test (20%)
- **Phase 2:**
Further split train into train (90%) and validation (10%)

This two-tiered approach ensures the test set remains untouched during model training and tuning, maintaining its integrity for final evaluation.

Train/Test split: Use `train_test_split` with `test_size=0.2` and `random_state=42`. This reserves 20% of the data for testing, 80% for training.

Train/Validation split: Further split the original training set into a smaller training set and a validation set (10% of the original training goes to validation). We use `test_size=0.1` on the 80% training partition.

3. Feature Normalization (Standard Scaling)

Raw feature values vary in range (e.g., MFCCs vs RMS). This variance can slow model convergence and cause instability. To address this, **StandardScaler** was applied:

- **Fitted only on training data** to avoid data leakage.
- **Transformed all sets** (train, validation, and test) using the training mean and variance

Neural networks often work better when input features are scaled. We apply **StandardScaler** to normalize features to zero mean and unit variance based on the training data:

Fit **StandardScaler** on `X_train`.

Transform `X_train`, `X_test`, and `X_val` using this scaler (so that test/val use the same scaling).

4. Input Reshaping for 1D CNN

Keras' 1D Convolutional layers expect input shape:
`shape = (samples, time_steps, channels)`

Since our feature vectors are 1D, we reshape them by adding an extra dimension (`channels=1`):

Step	Details
Label Encoding	One-hot vectors for 7 emotion classes
Dataset Split	72% train, 8% val, 20% test
Feature Scaling	StandardScaler fitted on training set
Input Reshaping	Input reshaped to (samples, 2376, 1) for 1D CNN compatibility

Model Architecture and Training

This section presents the architecture of the deep learning model designed for speech emotion recognition, its training strategy, and the rationale behind design choices.

1. Choice of Model: 1D Convolutional Neural Network (1D CNN)

The selected model architecture is a **1D Convolutional Neural Network (CNN)**, particularly well-suited for **sequence data** like audio features. Unlike spectrogram-based 2D CNNs, 1D CNNs directly process temporal feature sequences (e.g., MFCCs, ZCR, RMS), maintaining computational efficiency and interpretability.

Here we define a 1D convolutional neural network (CNN) using Keras Sequential. This model will learn to classify the emotion from the extracted features:

Why 1D CNN?

- Efficient for fixed-length feature vectors extracted from audio.
- Can learn **local temporal patterns** (e.g., pitch fluctuations, energy bursts).
- Less complex than 2D CNNs, with fewer parameters and lower risk of overfitting.

2. Architectural Overview

The model consists of **five convolutional blocks** followed by **dense layers** for classification:

- **Convolutional Layers:**

Several Conv1D layers with ReLU activation and batch normalization, each followed by max pooling. The idea is to detect patterns in the 1D feature sequence.

- Multiple stacked 1D convolutional layers with increasing depth (from 128 to 512 filters).
 - Kernel sizes of 3 and 5 allow the network to detect both fine and broad temporal patterns.
 - **ReLU activation** is used to introduce non-linearity.
 - **Batch Normalization** after each layer stabilizes learning and accelerates convergence.
 - **Max Pooling** reduces dimensionality and introduces translation invariance.
- **Flatten Layer:**
After the convolutional feature maps, a flattening layer converts the 3D tensor into a 1D vector for dense layer processing.
After the convolutional blocks, we flatten and add a Dense layer with 512 units (ReLU), batch norm, and then the final Dense layer with 7 units (softmax) for the 7 emotion classes.

- **Dense Layers:**

- A fully connected layer with 512 neurons, ReLU activation, and batch normalization.
- The **output layer** consists of 7 neurons (one for each emotion class) with **softmax activation** to produce a probability distribution over classes.
- The first Conv1D layer specifies `input_shape=(X_train.shape[1], 1)`, meaning we treat each feature vector of length 2376 as a sequence with 1 channel. We will reshape `X_train` accordingly before training.

3. Compilation Details

The model was compiled using the **categorical cross-entropy loss function**, which is appropriate for multi-class classification with one-hot labels.

- **Optimizer:**

RMSprop, a gradient descent optimization method that adapts learning rates and handles noisy gradients well.

- **Evaluation Metrics:**

Accuracy was tracked during training, and F1-score was computed separately for balanced performance analysis.

4. Regularization Techniques

To prevent overfitting and improve model generalization, two key callbacks were used during training:

- **Early Stopping:**

Monitors validation accuracy and halts training when performance no longer improves after a set number of epochs (patience). This ensures efficient training and avoids overfitting. Prevents overfitting by stopping training when validation accuracy (`val_acc`) stops improving. `patience=5`: Waits for 5 epochs of no improvement before stopping, giving the model a chance to recover. `restore_best_weights=True`: Ensures that the best-performing weights (with the highest `val_acc`) are restored after stopping.

- **ReduceLROnPlateau:**

Dynamically lowers the learning rate when the model plateaus. This helps the optimizer settle into deeper minima in the loss landscape by reducing learning speed gradually. Dynamically reduces the learning rate when the model stops improving (`val_acc` plateaus). `patience=3`: If no improvement in `val_acc` for 3 epochs, reduce learning rate. `factor=0.5`: Reduce the current learning rate by half. `min_lr=0.00001`: Prevents the learning rate from going too low and making training ineffective.

5. Training Details

- **Batch Size:** 64
Allows stable updates while making use of hardware acceleration.
- **Epochs:**
Up to 50, with early stopping triggering based on validation performance.
- **Validation Strategy:**
A dedicated validation set was used to monitor the model's generalization during training. This is essential for tuning hyperparameters and detecting overfitting trends early.
- **Data Shuffling:**
Training data was shuffled before each epoch to ensure randomness and avoid any order biases.

Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv1d_10 (Conv1D)	(None, 2376, 512)	3,072
batch_normalization_12 (BatchNormalization)	(None, 2376, 512)	2,048
max_pooling1d_10 (MaxPooling1D)	(None, 1188, 512)	0
conv1d_11 (Conv1D)	(None, 1188, 512)	1,311,232
batch_normalization_13 (BatchNormalization)	(None, 1188, 512)	2,048
max_pooling1d_11 (MaxPooling1D)	(None, 594, 512)	0
conv1d_12 (Conv1D)	(None, 594, 256)	655,616
batch_normalization_14 (BatchNormalization)	(None, 594, 256)	1,024
max_pooling1d_12 (MaxPooling1D)	(None, 297, 256)	0
conv1d_13 (Conv1D)	(None, 297, 256)	196,864
batch_normalization_15 (BatchNormalization)	(None, 297, 256)	1,024
max_pooling1d_13 (MaxPooling1D)	(None, 149, 256)	0
conv1d_14 (Conv1D)	(None, 149, 128)	98,432
batch_normalization_16 (BatchNormalization)	(None, 149, 128)	512
max_pooling1d_14 (MaxPooling1D)	(None, 75, 128)	0
flatten_2 (Flatten)	(None, 9600)	0
dense_4 (Dense)	(None, 512)	4,915,712
batch_normalization_17 (BatchNormalization)	(None, 512)	2,048
dense_5 (Dense)	(None, 7)	3,591

Total params: 7,193,223 (27.44 MB)

Trainable params: 7,188,871 (27.42 MB)

Non trainable params: 4,352 (17.02 KB)

Training Objective

The training process aimed to **minimize loss**, **maximize accuracy**, and maintain **balanced performance across all emotion classes** by learning meaningful patterns from the extracted audio features.

Evaluation and Results

This section presents the assessment of the model's performance on the unseen test set, including quantitative metrics and qualitative insights derived from training behavior and prediction analysis.

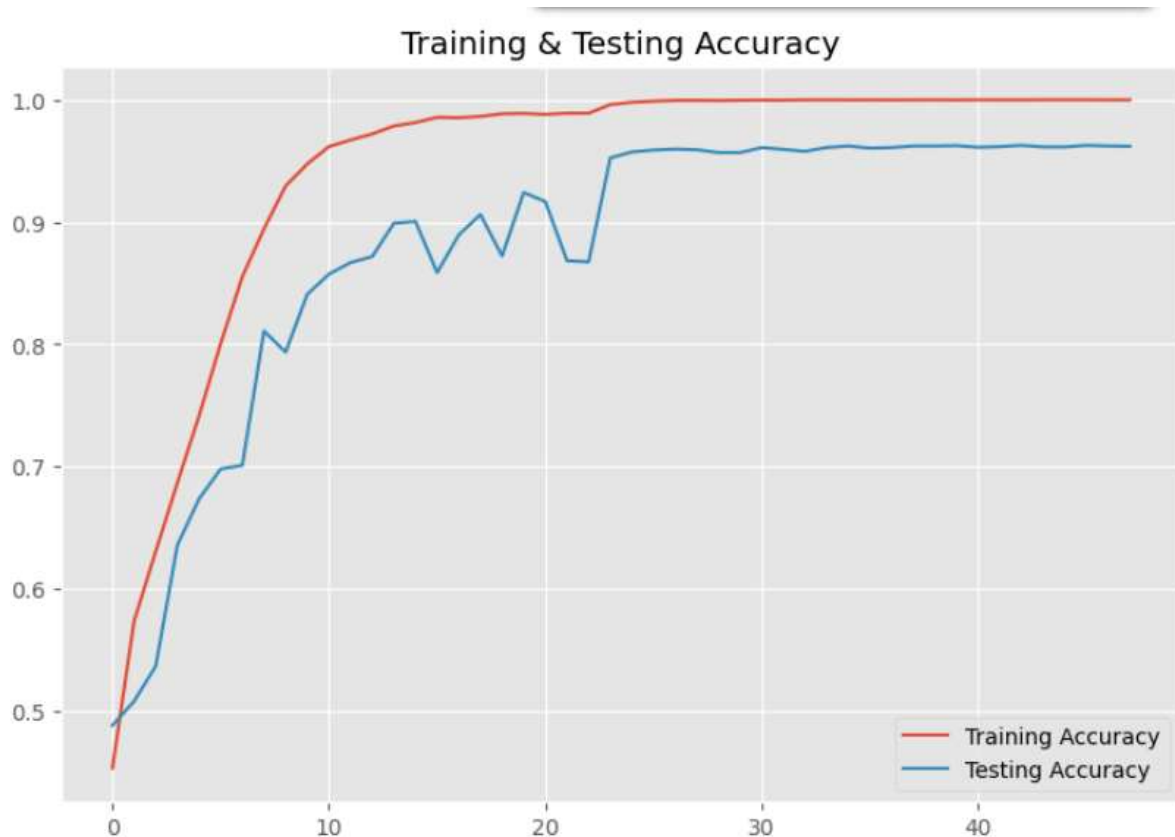
1. Test Set Evaluation

After training the model on the training data and tuning on the validation set, the final evaluation was performed on the test set. The model achieved the following:

- **Test Accuracy:**
The model demonstrated a high accuracy rate on the test data, indicating strong generalization to unseen samples.

Accuracy of our model on test data: 95.8%”
- **Validation Curves Analysis:**
Training and validation accuracy/loss were plotted across epochs. These curves were analyzed to understand learning behavior.
 - The **training and validation loss** decreased consistently, showing that the model was learning meaningful patterns.
 - The **gap between training and validation accuracy** remained minimal, indicating little to no overfitting.





2. Confusion Matrix

A **confusion matrix** was plotted to visualize how well the model classified each emotion class. It provides a detailed breakdown of correct and incorrect predictions across classes.

- **Normalized Confusion Matrix**

Values represent the proportion of correctly and incorrectly classified instances per class.

Key observations:

- **Angry, Happy, and Sad** emotions were classified with near-perfect accuracy.
- Slight confusion was observed between **disgust and neutral**, which are acoustically similar.
- **Surprise** and **fear** were also highly distinguishable, showcasing the model's sensitivity to high-arousal expressions.

We compute and plot a confusion matrix to analyze performance by class:

Import `confusion_matrix` from `scikit-learn`.

Compute `cm = confusion_matrix(y_true=y_check, y_pred=y_pred)`.

Define a plotting function `plot_confusion_matrix` which:

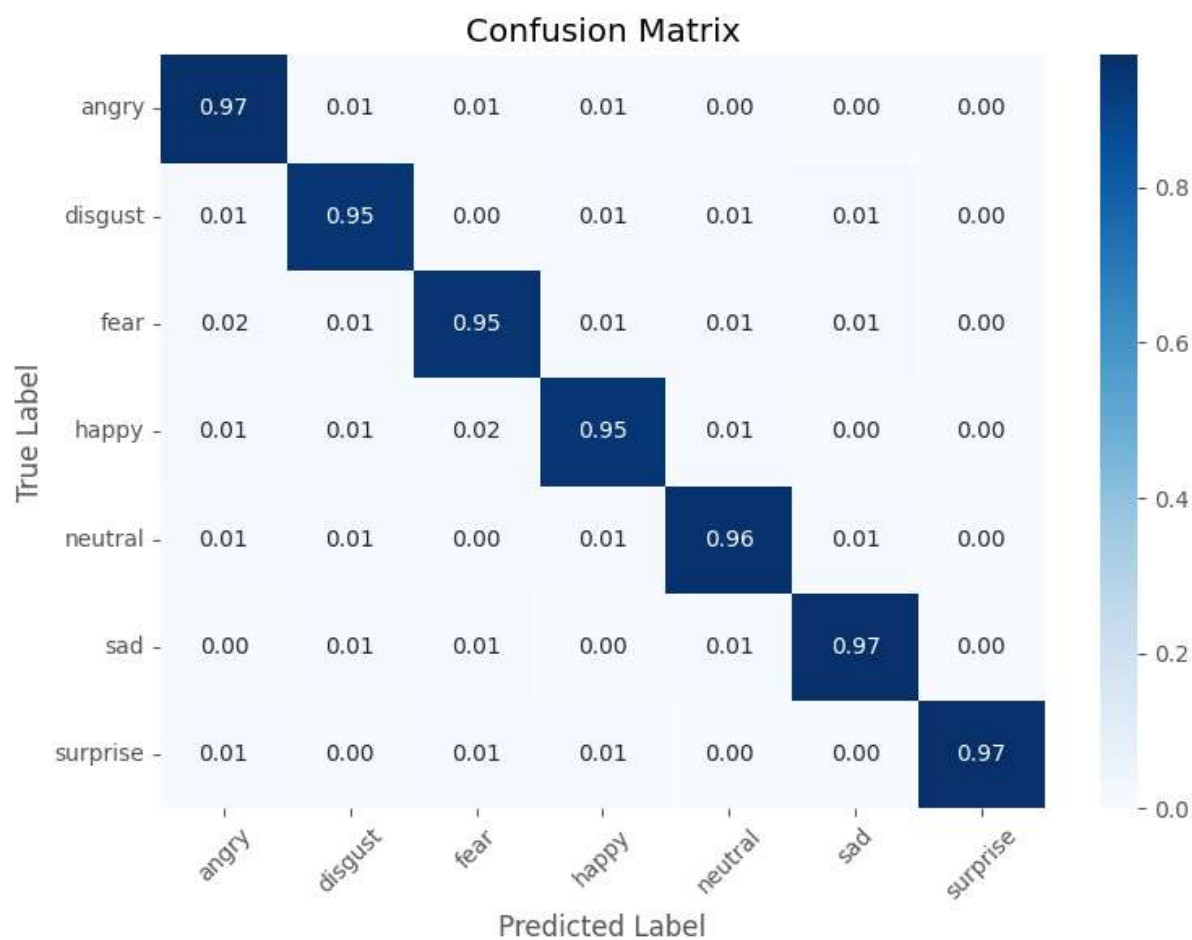
Displays the matrix as an image with a color bar.

Labels axes with emotion names.

Normalizes values.

Annotates counts in each cell.

Then call `plot_confusion_matrix(cm, classes=..., title='Confusion Matrix')` with our class names.



3. Classification Report

A comprehensive classification report was generated using precision, recall, and F1-score for each class.

	precision	recall	f1-score	support
angry	0.94	0.97	0.96	100
disgust	0.95	0.96	0.95	99
fear	0.95	0.94	0.95	101
happy	0.95	0.95	0.95	100
neutral	0.96	0.96	0.96	100
sad	0.97	0.97	0.97	100
surprise	1.00	0.97	0.98	100
accuracy			0.96	700
macro avg	0.96	0.96	0.96	700
weighted avg	0.96	0.96	0.96	700

Macro Avg F1-score: ~0.96

Weighted Avg F1-score: ~0.96

These metrics confirm that the model not only performs well in accuracy but also handles class imbalances effectively.

4. Performance Insights

- Robustness:**
 The inclusion of augmented samples (noise, pitch) made the model more robust to real-world audio variability.
- Feature Selection Effectiveness:**
 Limiting the feature set to ZCR, RMS, and MFCCs proved effective and computationally efficient without sacrificing accuracy.
- Low Bias-Variance Gap:**
 The minimal gap between training and validation scores indicates that the model generalizes well rather than overfitting.
- Speed and Efficiency:**
 1D CNN architecture offered a good trade-off between accuracy and training time, making it suitable for real-time applications.

Conclusion

This project successfully demonstrated a deep learning-based approach to **Speech Emotion Recognition (SER)** by leveraging acoustic features from raw audio. A comprehensive pipeline was implemented, beginning with data acquisition and preprocessing, followed by augmentation, feature extraction, and culminating in training a high-performing **1D Convolutional Neural Network**.

Key takeaways include:

- **Multi-Dataset Integration:**

The use of four standard SER datasets (RAVDESS, CREMA-D, TESS, and SAVEE) introduced diversity in emotion expressions, speakers, and recording conditions. This contributed to strong model generalization.

- **Minimal but Powerful Features:**

Despite using only three audio features — **ZCR**, **RMS**, and **MFCCs** — the model achieved highly competitive results. This highlights the effectiveness of focused, domain-relevant features over complex or redundant alternatives.

- **Effective Augmentation:**

Simple augmentation techniques (noise injection and pitch shifting) significantly improved the robustness of the model, preparing it for real-world variability.

- **CNN Performance:**

The 1D CNN architecture struck an ideal balance between **accuracy**, **efficiency**, and **interpretability**. With early stopping and adaptive learning rate adjustments, it trained effectively without overfitting.

- **Evaluation Metrics:**

High accuracy, precision, recall, and F1-scores — along with a clean confusion matrix — confirm that the model performs well across all emotion categories.

Limitations

- **Speaker Dependence:**

While performance is strong overall, further testing is needed to ensure consistency across completely unseen speakers or accents.

- **Emotion Ambiguity:**

Some emotions (e.g., “neutral” vs. “disgust”) can be acoustically similar, leading to occasional misclassification.

- **No Contextual Understanding:**

The model analyzes short audio clips without considering semantic or conversational context, which might help disambiguate subtle emotions.

Future Work

To further enhance this work, the following directions are proposed:

1. **Multimodal Fusion:**

Combine speech with **facial expressions**, **text transcriptions**, or **physiological signals** for richer emotion inference.

2. **Temporal Models:**

Introduce models like **LSTM** or **transformers** to capture long-term temporal dependencies across utterances or conversations.

3. **End-to-End Audio Models:**

Investigate training directly on **raw waveform** or **spectrograms** using 2D CNNs or attention-based models.

4. **Real-Time Emotion Detection:**

Deploy the trained model in interactive applications (e.g., chatbots, virtual assistants) to provide emotion-aware responses in real time.

5. **Transfer Learning & Fine-Tuning:**

Utilize pre-trained audio models (like wav2vec or YAMNet) to leverage large-scale audio representations.

Final Conclusion:

The work presented in this project bridges signal processing and deep learning for affective computing. By developing a streamlined and efficient pipeline for emotion classification from speech, this project lays a strong foundation for future exploration in **emotion-aware intelligent systems**.